

Ingeniería de Datos & SQL Masterclass

*El Tratado Definitivo de Diseño Relacional, Optimización de Consultas,
Arquitectura Cloud y Data Warehousing*

Serie de Arquitectura y Tecnologías de Bases de Datos

**Basado en el currículo de Ingeniería de Sistemas (ULima),
Patrones Prácticos de Producción y Benchmarks Cloud (AWS EC2 / Docker)**

Edición Profesional O'Reilly Media Standard

Ingeniería de Datos & SQL Masterclass

Manual Técnico de Referencia y Práctica Industrial

Formato y compilación editorial inspirada en el estándar internacional O'Reilly.

Contenido académico: Facultad de Ingeniería de Sistemas — Universidad de Lima.

Datasets de referencia: *Northwind Traders (PostgreSQL 16)*, *Cineplex*, *GameHub*, *Gourmet*.

Índice general

I	Fundamentos y Modelamiento Relacional	3
1.	Fundamentos de Administración de Información y Ciclo de Vida del Dato	4
1.1.	Marco Conceptual y Teórico (Currículo Universitario)	4
1.1.1.	Dato, Información y Conocimiento (Pirámide DIKW)	4
1.1.2.	Evolución: De Sistemas Tradicionales de Archivos a DBMS	4
1.2.	Casos de Estudio y Práctica Aplicada	5
1.2.1.	Definición de Entornos y Metadatos	5
1.3.	Taller de Consolidación del Capítulo	5
1.3.1.	Solución Técnica en PostgreSQL 16	5
2.	Arquitectura de DBMS, Independencia de Datos y Transacciones ACID	7
2.1.	Marco Conceptual y Teórico (Currículo Universitario)	7
2.1.1.	Arquitectura ANSI/SPARC de Tres Niveles	7
2.1.2.	Independencia de Datos	7
2.1.3.	El Modelo Transaccional ACID	7
2.2.	Casos de Estudio: Control Concurrente y Bloques Transaccionales	8
2.3.	Taller de Consolidación del Capítulo	8
2.3.1.	Solución Técnica en PostgreSQL	8
3.	Bases de Datos Relacionales, Integridad Formal y Lenguaje DDL	10
3.1.	Marco Conceptual y Teórico (Currículo Universitario)	10
3.1.1.	Fundamentos del Modelo Relacional	10
3.1.2.	Restricciones de Integridad Formales	10
3.2.	Casos de Estudio y Práctica Aplicada	10
3.3.	Taller de Consolidación del Capítulo	11
3.3.1.	Solución Técnica en PostgreSQL / MySQL	11
4.	Manipulación de Datos (DML), Filtrado y Consultas Fundamentales	13
4.1.	Marco Conceptual y Teórico (Currículo Universitario)	13
4.1.1.	Lógica de Tres Valores (3VL) y Tratamiento de NULLs	13
4.2.	Casos de Estudio y Práctica Aplicada	13
4.3.	Taller de Consolidación del Capítulo	14
4.3.1.	Solución Técnica en PostgreSQL	14
5.	Modelamiento Conceptual Entidad-Relación (DER/EER) y Cardinalidad	15
5.1.	Marco Conceptual y Teórico (Currículo Universitario)	15
5.1.1.	El Modelo Entidad-Relación Extendido (EER)	15
5.2.	Casos de Estudio: Mapeo de Relaciones N:M a Tablas Puente	15
5.3.	Taller de Consolidación del Capítulo	16
5.3.1.	Solución Técnica Relacional	16

6. Normalización Formal de Bases de Datos (1FN a 3FN) y Anomalías	18
6.1. Marco Conceptual y Teórico (Currículo Universitario)	18
6.1.1. Dependencias Funcionales	18
6.1.2. Las Tres Formas Normales Clásicas	18
6.2. Casos de Estudio y Descomposición sin Pérdida	18
6.2.1. Descomposición Formal en 3FN	18
 II Consultas Avanzadas, Álgebra de JOINS y SQL Analítico	 20
7. Formas Normales Superiores (BCNF, 4FN, 5FN, DKNF) y Desnormalización	21
7.1. Marco Conceptual y Teórico (Currículo Universitario)	21
7.1.1. Forma Normal de Boyce-Codd (BCNF)	21
7.1.2. Cuarta Forma Normal (4FN) y Dependencias Multivaluadas	21
7.1.3. Quinta Forma Normal (5FN / PJ-NF) y DKNF	21
7.1.4. Desnormalización Estratégica y Rendimiento	22
7.2. Casos de Estudio y Práctica Aplicada	22
7.3. Taller de Consolidación del Capítulo	22
7.3.1. Solución Técnica en PostgreSQL	23
 8. Álgebra Relacional de JOINS, Subconsultas y CTEs	 24
8.1. Marco Conceptual y Teórico (Currículo Universitario)	24
8.1.1. Familias de Operadores JOIN	24
8.2. Taller de Consolidación del Capítulo	24
8.2.1. Solución Técnica en PostgreSQL con CTEs y LEFT JOIN	24
 9. SQL Analítico Avanzado: Funciones de Ventana y Agregación Multidimensional	 26
9.1. Marco Conceptual y Teórico (Currículo Universitario)	26
9.1.1. Sintaxis Formal: Cláusula OVER()	26
9.1.2. Familias de Funciones Analíticas	26
9.2. Taller de Consolidación del Capítulo	26
9.2.1. Solución Técnica en PostgreSQL (Formato O'Reilly)	27
 10. Programación en Base de Datos: PL/pgSQL, Procedimientos y Triggers	 28
10.1. Marco Conceptual y Teórico (Currículo Universitario)	28
10.1.1. Estructura de Bloques en PL/pgSQL	28
10.1.2. Disparadores (Triggers) y Eventos	28
10.2. Casos de Estudio y Práctica Aplicada	29
10.3. Taller de Consolidación del Capítulo	30
10.3.1. Solución Técnica con Trigger en PostgreSQL	30
 III Data Warehousing, Rendimiento y Alta Disponibilidad	 32
11. Data Warehousing, Modelado Dimensional y Arquitectura OLAP	33
11.1. Marco Conceptual y Teórico (Currículo Universitario)	33
11.1.1. Comparativa OLTP vs OLAP	33
11.1.2. Metodología Kimball: Hechos y Dimensiones	33
11.2. Casos de Estudio: Operaciones OLAP (CUBE y ROLLUP)	34
11.3. Taller de Consolidación del Capítulo	34
11.3.1. Solución Técnica DDL Dimensional	34

12.Optimización de Consultas, Planes de Ejecución e Índices	36
12.1. Marco Conceptual y Teórico (Currículo Universitario)	36
12.1.1. Componentes de un Plan de Ejecución	36
12.1.2. Estrategias de Indexación Avanzada	36
12.2. Casos de Estudio y Práctica Aplicada	37
12.3. Taller de Consolidación del Capítulo	37
12.3.1. Solución Técnica y Estrategia de Tuning	37
13.Alta Disponibilidad (HA), NoSQL y Arquitecturas Cloud Modernas	39
13.1. Marco Conceptual y Teórico (Currículo Universitario)	39
13.1.1. El Teorema CAP y Bases NoSQL	39
13.1.2. Arquitecturas de Alta Disponibilidad en RDBMS	39
13.2. Casos de Estudio: Almacenamiento Semiestructurado con JSONB y GIN . . .	39
13.3. Taller de Consolidación del Capítulo	40
13.3.1. Solución Técnica con Índices de Expresión y GIN	40
 IV Laboratorios de Consolidación e Infraestructura	 42
14.Infraestructura Cloud, Despliegue en AWS EC2 y Benchmark Northwind	43
14.1. Aprovisionamiento de Infraestructura en AWS EC2	43
14.1.1. Manifiesto de Orquestación Docker Compose	43
14.2. Batería de Ejercicios Graduales y Solucionario Técnico	44
14.2.1. Solución Técnica en PostgreSQL	44
15.Taller Multi-Motor y Casos de Negocio (PostgreSQL / MySQL)	45
15.1. Desarrollo Práctico Multi-Motor y Compatibilidad	45
15.1.1. Comparativa de Sintaxis y Dialectos	45
15.2. Casos de Negocio Reales	45
15.2.1. Caso 1: Cineplex (Gestión de Salas y Boletos)	45

Parte I

**Fundamentos y Modelamiento
Relacional**

Capítulo 1

Fundamentos de Administración de Información y Ciclo de Vida del Dato

1.1. Marco Conceptual y Teórico (Currículo Universitario)

En el desarrollo de software y la ingeniería de sistemas moderna, el manejo riguroso de la información constituye la base de la toma de decisiones empresariales.

1.1.1. Dato, Información y Conocimiento (Pirámide DIKW)

- **Dato:** Hecho aislado, representación simbólica no estructurada o valor bruto (ej. 42, "2026-08-22").
- **Información:** Conjunto de datos contextualizados, estructurados y con significado semántico para un receptor.
- **Conocimiento:** Asimilación de la información combinada con reglas de negocio, patrones empíricos y experiencia.
- **Sabiduría:** Aplicación estratégica del conocimiento para optimizar procesos organizacionales.

1.1.2. Evolución: De Sistemas Tradicionales de Archivos a DBMS

Históricamente, los sistemas dependientes del sistema de archivos del sistema operativo sufrían de:

1. **Redundancia e Inconsistencia de Datos:** Duplicación innecesaria de atributos en diferentes ficheros.
2. **Dificultad de Acceso y Aislamiento de Datos:** Necesidad de compilar un nuevo programa ejecutable para cada consulta ad-hoc.
3. **Anomalías de Acceso Concurrente:** Riesgo de escrituras superpuestas y corrupción de estado.
4. **Problemas de Seguridad e Integridad:** Carencia de mecanismos declarativos para forzar restricciones lógicas.

Un **Sistema de Gestión de Bases de Datos (DBMS)** resuelve estas limitaciones centralizando el acceso, garantizando la consistencia y abstrayendo la representación física.

1.2. Casos de Estudio y Práctica Aplicada

Para ilustrar la transición de datos no estructurados a esquemas relacionales gobernados, consideremos la gestión de personal y órdenes comerciales.

1.2.1. Definición de Entornos y Metadatos

En entornos empresariales, cada entidad debe tener un tipo de dato estricto y restricciones no nulas:

Listing 1.1: Definición inicial de estructura de auditoría

```

1  -- Esquema base de registro de metadatos de auditoría
2  CREATE TABLE audit_log (
3      log_id BIGSERIAL PRIMARY KEY,
4      entity_name VARCHAR(50) NOT NULL,
5      record_id BIGINT NOT NULL,
6      action_type VARCHAR(10) NOT NULL CHECK (action_type IN ('INSERT',
7          'UPDATE', 'DELETE')),
8      changed_by VARCHAR(100) DEFAULT CURRENT_USER,
9      changed_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
10     payload JSONB
11 );

```

1.3. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Una empresa internacional de comercio minorista necesita calcular la antigüedad exacta de todos sus colaboradores y clasificar automáticamente a su personal según su rango de permanencia en la organización para la asignación de bonificaciones corporativas, evitando inconsistencias temporales producidas por zonas horarias.

1.3.1. Solución Técnica en PostgreSQL 16

Listing 1.2: Consulta optimizada de cálculo temporal de personal

```

1  -- Soluci\on en PostgreSQL con tipado estricto y formato O'Reilly
2  SELECT
3      employee_id,
4      first_name || ' ' || last_name AS full_name,
5      hire_date,
6      EXTRACT(YEAR FROM AGE(CURRENT_DATE, hire_date))::INTEGER AS
7          years_of_service,
8      CASE
9          WHEN EXTRACT(YEAR FROM AGE(CURRENT_DATE, hire_date)) >= 10
10             THEN 'Senior Veteran'
11          WHEN EXTRACT(YEAR FROM AGE(CURRENT_DATE, hire_date)) >= 5
12             THEN 'Mid-Level'
13          ELSE 'Junior / Nuevo Ingreso'
14      END AS career_tier
15  FROM employees
16  WHERE hire_date IS NOT NULL
17  ORDER BY
18      years_of_service DESC,

```

```
16 last_name ASC;
```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Uso de AGE(CURRENT_DATE, hire_date):** En PostgreSQL, la función nativa AGE() computa la diferencia exacta en años, meses y días considerando años bisiestos y longitudes variables de meses, superando las restas aritméticas simples de 365 días.
- **Proyección Explícita:** Se omitió SELECT * para permitir al optimizador usar índices de solo escaneo (*Index-Only Scan*) en caso de que existan índices en (hire_date, last_name).

Capítulo 2

Arquitectura de DBMS, Independencia de Datos y Transacciones ACID

2.1. Marco Conceptual y Teórico (Currículo Universitario)

2.1.1. Arquitectura ANSI/SPARC de Tres Niveles

El estándar ANSI/SPARC define tres niveles de abstracción esenciales para desacoplar las aplicaciones de la persistencia física:

1. **Nivel Externo (Vistas de Usuario):** Describe la parte de la base de datos relevante para un grupo de usuarios particular mediante vistas y restricciones de acceso.
2. **Nivel Conceptual (Esquema Lógico):** Modela la estructura lógica global (tablas, llaves primarias, llaves foráneas y reglas de integridad de negocio).
3. **Nivel Interno (Esquema Físico):** Define cómo se distribuyen los bloques en disco, estructuras de almacenamiento (Heap files, B-Trees) y optimización de I/O.

2.1.2. Independencia de Datos

- **Independencia Lógica:** Capacidad de modificar el esquema conceptual (añadir nuevas columnas o tablas) sin alterar las aplicaciones existentes ni las vistas externas.
- **Independencia Física:** Capacidad de reorganizar el almacenamiento interno (añadir índices, cambiar métodos de acceso o particionar tablas) sin alterar el esquema conceptual.

2.1.3. El Modelo Transaccional ACID

- **Atomicidad (A):** La transacción se ejecuta como una unidad indivisible: o se completan todas las operaciones (COMMIT) o se revierten por completo (ROLLBACK).
- **Consistencia (C):** La transacción transforma la base de datos de un estado válido a otro, cumpliendo todas las restricciones de integridad.
- **Aislamiento (I):** Las transacciones concurrentes operan sin interferencias no controladas (Niveles: *Read Uncommitted*, *Read Committed*, *Repeatable Read*, *Serializable*).
- **Durabilidad (D):** Una vez confirmado el commit, los cambios persisten incluso ante caídas del servidor gracias al registro de transacciones (*Write-Ahead Logging / WAL*).

2.2. Casos de Estudio: Control Concurrente y Bloques Transaccionales

Listing 2.1: Manejo seguro de transacciones con bloqueo explícito

```
1  -- Bloque transaccional seguro con actualizaci\on at\omica de
   inventario
2  BEGIN;
3
4  -- Bloqueo pesimista a nivel de fila para evitar condiciones de
   carrera (Race Conditions)
5  SELECT
6      product_id,
7      product_name,
8      units_in_stock
9  FROM products
10 WHERE product_id = 11
11 FOR UPDATE;
12
13 -- Actualizaci\on condicional segura
14 UPDATE products
15 SET units_in_stock = units_in_stock - 5
16 WHERE product_id = 11
17     AND units_in_stock >= 5;
18
19 COMMIT;
```

2.3. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Una plataforma de comercio electrónico necesita procesar la cancelación de un pedido de compra, reintegrando automáticamente el stock físico a las tablas de inventario y marcando la orden como anulada en una sola transacción indivisible para evitar desajustes de auditoría.

2.3.1. Solución Técnica en PostgreSQL

Listing 2.2: Transacción ACID de anulación y restitución de inventario

```
1  -- Transacci\on segura con restituci\on autom\atica y RETURNING
2  BEGIN;
3
4  -- 1. Actualizar estado de la orden
5  UPDATE orders
6  SET
7      shipped_date = NULL,
8      ship_name = 'ANULADO - ' || ship_name
9  WHERE order_id = 10248
10 RETURNING order_id, customer_id;
11
```

```
12 -- 2. Restituir el stock de cada producto involucrado
13 UPDATE products AS p
14 SET units_in_stock = p.units_in_stock + od.quantity
15 FROM order_details AS od
16 WHERE od.order_id = 10248
17     AND p.product_id = od.product_id;
18
19 COMMIT;
```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Integridad de Transacción:** Al encerrar ambas sentencias entre BEGIN y COMMIT, si la base de datos se reinicia o falla durante el segundo UPDATE, PostgreSQL ejecuta un ROLLBACK automático protegiendo el balance contable.
- **Uso de RETURNING:** Elimina la necesidad de ejecutar un SELECT previo para confirmar el identificador procesado.

Capítulo 3

Bases de Datos Relacionales, Integridad Formal y Lenguaje DDL

3.1. Marco Conceptual y Teórico (Currículo Universitario)

3.1.1. Fundamentos del Modelo Relacional

Propuesto por Edgar F. Codd (1970), el modelo se sustenta en la teoría matemática de conjuntos y la lógica de predicados de primer orden:

- **Relación:** Una tabla bidimensional de datos donde el orden de filas y columnas es irrelevante.
- **Tupla:** Una fila individual en la relación que representa una instancia única de la entidad.
- **Atributo:** Columna con nombre y tipo de dato formal.
- **Dominio:** Conjunto finito o infinito de valores atómicos permitidos para un atributo.

3.1.2. Restricciones de Integridad Formales

1. **Integridad de Entidad:** Toda tabla debe poseer una Clave Primaria (*Primary Key*) única y no nula (NOT NULL).
2. **Integridad Referencial:** Toda Clave Foránea (*Foreign Key*) debe coincidir con un valor existente de clave primaria en la tabla referenciada o ser nula.
3. **Integridad de Dominio:** Los valores deben adherirse a las restricciones de tipo y chequeos (CHECK).

3.2. Casos de Estudio y Práctica Aplicada

Listing 3.1: Definición DDL con restricciones de integridad completas

```
1 -- Creación de esquema robusto para salas de cine y funciones
2 CREATE TABLE cinemas (
3     cinema_id SERIAL PRIMARY KEY,
4     cinema_name VARCHAR(100) NOT NULL,
5     city VARCHAR(50) NOT NULL,
6     total_screens SMALLINT DEFAULT 1 CHECK (total_screens > 0),
```

```

7      is_active BOOLEAN DEFAULT TRUE,
8      created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
9  );
10
11 CREATE TABLE movie_showtimes (
12     showtime_id BIGSERIAL PRIMARY KEY,
13     cinema_id INTEGER NOT NULL,
14     movie_title VARCHAR(150) NOT NULL,
15     start_time TIMESTAMPTZ NOT NULL,
16     ticket_price NUMERIC(8, 2) NOT NULL CHECK (ticket_price >= 0.00),
17     CONSTRAINT fk_showtimes_cinema
18         FOREIGN KEY (cinema_id)
19         REFERENCES cinemas (cinema_id)
20         ON DELETE CASCADE
21         ON UPDATE CASCADE
22 );
23
24 -- \'indice para optimizaci\'on de b\'usqueda por horario y cine
25 CREATE INDEX idx_showtimes_cinema_time ON movie_showtimes (cinema_id,
    start_time);

```

3.3. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Implementar un módulo DDL para una tienda de videojuegos en línea, asegurando que los descuentos no superen el 100 % del valor unitario del producto y que las categorías mantengan nombres únicos en minúsculas.

3.3.1. Solución Técnica en PostgreSQL / MySQL

Listing 3.2: Script DDL con constraints y tipos personalizados

```

1 CREATE TABLE game_categories (
2     category_id SERIAL PRIMARY KEY,
3     category_slug VARCHAR(50) NOT NULL UNIQUE,
4     display_name VARCHAR(80) NOT NULL,
5     CONSTRAINT chk_category_slug_lowercase CHECK (category_slug =
        LOWER(category_slug))
6 );
7
8 CREATE TABLE video_games (
9     game_id BIGSERIAL PRIMARY KEY,
10    category_id INTEGER NOT NULL REFERENCES
        game_categories(category_id),
11    title VARCHAR(120) NOT NULL,
12    base_price NUMERIC(10, 2) NOT NULL CHECK (base_price >= 0),
13    discount_percentage NUMERIC(5, 2) DEFAULT 0.00 CHECK
        (discount_percentage BETWEEN 0.00 AND 100.00),
14    release_date DATE NOT NULL,
15    is_available BOOLEAN DEFAULT TRUE
16 );

```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Restricciones de Dominio con CHECK:** Garantizan que datos corruptos o valores negativos jamás ingresen al motor de almacenamiento, independientemente del lenguaje o framework utilizado en el backend.
- **ON DELETE CASCADE:** Debe usarse con precaución extrema en entornos de producción financiera; en esquemas transaccionales se prefiere **ON DELETE RESTRICT** o borrado lógico mediante banderas (*soft delete*).

Capítulo 4

Manipulación de Datos (DML), Filtrado y Consultas Fundamentales

4.1. Marco Conceptual y Teórico (Currículo Universitario)

El Lenguaje de Manipulación de Datos (DML) permite interactuar con el estado interno de las relaciones relacionales mediante operaciones de inserción, actualización, eliminación y consulta (DQL).

4.1.1. Lógica de Tres Valores (3VL) y Tratamiento de NULLs

En SQL, una condición lógica puede evaluarse en: **TRUE**, **FALSE** o **UNKNOWN**. Los valores NULL representan la ausencia de valor o valor desconocido:

- La expresión `columna = NULL` siempre evalúa a **UNKNOWN**.
- Se debe usar obligatoriamente el predicado formal `IS NULL` o `IS NOT NULL`.
- En ordenamientos, PostgreSQL coloca los NULL al final por defecto en orden descendente (`NULLS LAST`).

4.2. Casos de Estudio y Práctica Aplicada

Listing 4.1: Filtrado riguroso de pedidos con funciones escalares y fechas

```
1  -- Consulta analítica de pedidos internacionales con cálculo de
   d'ías de demora
2  SELECT
3      order_id,
4      customer_id,
5      order_date,
6      shipped_date,
7      (shipped_date - order_date) AS days_to_ship,
8      freight,
9      COALESCE(ship_region, 'Sin Región Asignada') AS region_status
10 FROM orders
11 WHERE order_date >= '1997-01-01'::DATE
12       AND shipped_date IS NOT NULL
13       AND freight > 50.00
14 ORDER BY
15     freight DESC,
```

```

16     days_to_ship ASC
17 LIMIT 20;

```

4.3. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

El área de logística de Northwind requiere identificar todos los productos descontinuados o con stock crítico (menos de 10 unidades en almacén pero con pedidos pendientes de clientes), calculando el valor total monetario inmovilizado en inventario.

4.3.1. Solución Técnica en PostgreSQL

Listing 4.2: Consulta de evaluación de riesgo de stock e inventario

```

1  SELECT
2      product_id,
3      product_name,
4      unit_price,
5      units_in_stock,
6      units_on_order,
7      (unit_price * units_in_stock)::NUMERIC(12, 2) AS
8          total_inventory_value,
9      CASE
10         WHEN discontinued = 1 THEN 'DESCONTINUADO'
11         WHEN units_in_stock = 0 THEN 'ROTURA TOTAL DE STOCK'
12         WHEN units_in_stock < 10 THEN 'ALERTA: STOCK CRÍTICO'
13         ELSE 'STOCK NORMAL'
14     END AS inventory_health_status
15 FROM products
16 WHERE discontinued = 1
17    OR (units_in_stock < 10 AND units_on_order > 0)
18 ORDER BY
19     total_inventory_value DESC;

```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Casting Explícito:** El uso de `::NUMERIC(12, 2)` previene errores de precisión flotante (*floating-point inaccuracies*) inherentes a tipos como `REAL` o `FLOAT`.
- **Función COALESCE:** Permite sustituir de forma segura valores desconocidos o nulos sin alterar el tipo de retorno.

Capítulo 5

Modelamiento Conceptual Entidad-Relación (DER/EER) y Cardinalidad

5.1. Marco Conceptual y Teórico (Currículo Universitario)

5.1.1. El Modelo Entidad-Relación Extendido (EER)

El modelamiento conceptual traduce los requerimientos informales de negocio en especificaciones abstractas no dependientes de ningún motor físico específico.

- **Entidades Fuertes y Débiles:** Las entidades fuertes poseen existencia independiente y clave propia; las entidades débiles dependen existencialmente de otra entidad y se identifican mediante una clave parcial más la clave foránea del propietario.
- **Tipos de Atributos:**
 - *Simples / Atómicos:* No divisibles (ej. `precio`).
 - *Compuestos:* Jerárquicos (ej. `Direccion: Calle, Ciudad, CP`).
 - *Multivaluados:* Colecciones de valores para una misma instancia (ej. `telefonos[]`).
 - *Derivados:* Calculados en tiempo de ejecución (ej. `edad = CURRENT_DATE - fecha_nacimiento`).
- **Cardinalidad y Participación:** Ratios estructurales ($1 : 1$, $1 : N$, $N : M$) y participación total (obligatoria) o parcial (opcional).

5.2. Casos de Estudio: Mapeo de Relaciones N:M a Tablas Puente

Cuando existe una relación Muchos a Muchos entre entidades de negocio (por ejemplo, **órdenes** y **Productos**), la teoría relacional exige la creación de una tabla asociativa con clave primaria compuesta:

Listing 5.1: Implementación física de la tabla puente Order Details

```
1 -- Tabla asociativa que resuelve la relación N:M entre Orders y
  Products
2 CREATE TABLE order_details (
```

```

3      order_id SMALLINT NOT NULL REFERENCES orders(order_id) ON DELETE
        CASCADE,
4      product_id SMALLINT NOT NULL REFERENCES products(product_id),
5      unit_price REAL NOT NULL CHECK (unit_price > 0),
6      quantity SMALLINT NOT NULL CHECK (quantity > 0),
7      discount REAL DEFAULT 0.0 CHECK (discount >= 0 AND discount <= 1),
8      PRIMARY KEY (order_id, product_id)
9 );

```

5.3. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Diseñar el esquema de base de datos relacional para un sistema de reservas de restaurantes gourmet, contemplando mesas, meseros asignados por turno, cartas de menú con platillos categorizados y comandas de mesas con múltiples platos solicitados.

5.3.1. Solución Técnica Relacional

Listing 5.2: Esquema DDL completo para sistema de restaurantes

```

1  -- 1. Tablas Maestras
2  CREATE TABLE restaurant_tables (
3      table_id SERIAL PRIMARY KEY,
4      table_number VARCHAR(10) NOT NULL UNIQUE,
5      seating_capacity SMALLINT NOT NULL CHECK (seating_capacity > 0),
6      location_area VARCHAR(30) DEFAULT 'Salon Principal'
7  );
8
9  CREATE TABLE waitstaff (
10     staff_id SERIAL PRIMARY KEY,
11     first_name VARCHAR(50) NOT NULL,
12     last_name VARCHAR(50) NOT NULL,
13     shift_type VARCHAR(20) CHECK (shift_type IN ('Manana', 'Tarde',
14         'Noche'))
15 );
16
17 CREATE TABLE menu_dishes (
18     dish_id SERIAL PRIMARY KEY,
19     dish_name VARCHAR(100) NOT NULL,
20     category VARCHAR(40) NOT NULL,
21     price NUMERIC(8, 2) NOT NULL CHECK (price > 0)
22 );
23
24 -- 2. Entidad Transaccional y Detalle
25 CREATE TABLE restaurant_orders (
26     order_id BIGSERIAL PRIMARY KEY,
27     table_id INTEGER NOT NULL REFERENCES restaurant_tables(table_id),
28     staff_id INTEGER NOT NULL REFERENCES waitstaff(staff_id),
29     opened_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
30     closed_at TIMESTAMPTZ,
31     status VARCHAR(20) DEFAULT 'ABIERTA' CHECK (status IN ('ABIERTA',
32         'PAGADA', 'CANCELADA'))
33 );

```

```
32
33 CREATE TABLE order_dish_items (
34     item_id BIGSERIAL PRIMARY KEY,
35     order_id BIGINT NOT NULL REFERENCES restaurant_orders(order_id)
36         ON DELETE CASCADE,
37     dish_id INTEGER NOT NULL REFERENCES menu_dishes(dish_id),
38     quantity SMALLINT NOT NULL DEFAULT 1 CHECK (quantity > 0),
39     special_notes TEXT
39 );
```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Claves Primarias Compuestas vs Sustitutas:** En tablas intermedias con atributos propios (*payload*) o con posibilidad de repetición del mismo ítem en una comanda, es aconsejable emplear una clave sustituta (**BIGSERIAL**) manteniendo un índice único en caso de requerir no-duplicidad.

Capítulo 6

Normalización Formal de Bases de Datos (1FN a 3FN) y Anomalías

6.1. Marco Conceptual y Teórico (Currículo Universitario)

La normalización es la técnica formal de diseño lógico de bases de datos relacionales orientada a minimizar la redundancia y eliminar las anomalías de actualización, inserción y borrado.

6.1.1. Dependencias Funcionales

Dado un esquema R , un conjunto de atributos Y depende funcionalmente de X ($X \rightarrow Y$) si y solo si cada valor de X en R se asocia con exactamente un único valor de Y .

6.1.2. Las Tres Formas Normales Clásicas

1. **Primera Forma Normal (1FN):** Todos los atributos deben contener únicamente valores atómicos e indivisibles (sin grupos repetitivos ni arrays no normalizados).
2. **Segunda Forma Normal (2FN):** Cumple la 1FN y ningún atributo no primo depende parcialmente de una clave candidata compuesta (eliminación de dependencia funcional parcial).
3. **Tercera Forma Normal (3FN):** Cumple la 2FN y ningún atributo no primo depende transitivamente de la clave primaria ($X \rightarrow Y \rightarrow Z$).

6.2. Casos de Estudio y Descomposición sin Pérdida

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Se dispone de un formulario desnormalizado de órdenes de venta donde los datos del cliente, transportista y detalle del producto residen en una sola estructura, generando duplicidad de registros cada vez que un cliente realiza un nuevo pedido.

6.2.1. Descomposición Formal en 3FN

Listing 6.1: Estructuras normalizadas en 3FN

```
1  -- 1. Tabla de Clientes (Elimina dependencia transitiva)
2  CREATE TABLE customers (
3      customer_id BPCHAR(5) PRIMARY KEY,
4      company_name VARCHAR(40) NOT NULL,
5      contact_name VARCHAR(30),
6      country VARCHAR(15)
7  );
8
9  -- 2. Tabla de \órdenes (Depende directamente del cliente)
10 CREATE TABLE orders (
11     order_id SMALLINT PRIMARY KEY,
12     customer_id BPCHAR(5) NOT NULL REFERENCES customers(customer_id),
13     order_date DATE NOT NULL,
14     freight REAL DEFAULT 0.0
15 );
16
17 -- 3. Tabla de Productos (Independiente del pedido)
18 CREATE TABLE products (
19     product_id SMALLINT PRIMARY KEY,
20     product_name VARCHAR(40) NOT NULL,
21     unit_price REAL NOT NULL
22 );
23
24 -- 4. Detalle de \órdenes (Relaci\on N:M normalizada)
25 CREATE TABLE order_details (
26     order_id SMALLINT NOT NULL REFERENCES orders(order_id),
27     product_id SMALLINT NOT NULL REFERENCES products(product_id),
28     unit_price REAL NOT NULL,
29     quantity SMALLINT NOT NULL,
30     discount REAL DEFAULT 0.0,
31     PRIMARY KEY (order_id, product_id)
32 );
```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Preservación de Dependencias:** La descomposición garantiza que todas las restricciones de integridad y dependencias funcionales puedan verificarse localmente en cada tabla individual sin requerir uniones complejas antes de cada inserción.

Parte II

Consultas Avanzadas, álgebra de JOINS y SQL Analítico

Capítulo 7

Formas Normales Superiores (BCNF, 4FN, 5FN, DKNF) y Desnormalización

7.1. Marco Conceptual y Teórico (Currículo Universitario)

A medida que los requerimientos de modelamiento se vuelven más complejos, la Tercera Forma Normal (3FN) resulta insuficiente para prevenir ciertas redundancias causadas por claves candidatas superpuestas o dependencias multivaluadas.

7.1.1. Forma Normal de Boyce-Codd (BCNF)

Una relación R está en BCNF si y solo si, para cada dependencia funcional no trivial $X \rightarrow Y$, X es una **superclave** de R .

La diferencia crítica con la 3FN radica en que la 3FN permite que Y sea un atributo primo (parte de alguna clave candidata), mientras que BCNF exige estrictamente que el determinante X sea una superclave completa.

7.1.2. Cuarta Forma Normal (4FN) y Dependencias Multivaluadas

Una dependencia multivaluada $X \twoheadrightarrow Y$ indica que para un valor dado de X , existe un conjunto bien definido de valores de Y que es completamente independiente de los demás atributos en la relación.

- **Definición de 4FN:** Una relación está en 4FN si cumple BCNF y para toda dependencia multivaluada no trivial $X \twoheadrightarrow Y$, X es una superclave.

7.1.3. Quinta Forma Normal (5FN / PJ-NF) y DKNF

- **5FN (Project-Join Normal Form):** Trata sobre dependencias de unión (*Join Dependencies*). Una relación está en 5FN si no puede descomponerse en subrelaciones sin generar tuplas espurias al reconstruirla mediante operaciones **NATURAL JOIN**.
- **Forma Normal Dominio-Clave (DKNF):** Es el límite teórico máximo de la normalización. Una relación está en DKNF si toda restricción de integridad puede deducirse directamente de las definiciones de dominio y de claves.

7.1.4. Desnormalización Estratégica y Rendimiento

En entornos analíticos y de alta concurrencia de lectura (OLAP), la normalización extrema incrementa la sobrecarga de computación debido al excesivo encadenamiento de JOINS. La desnormalización controlada introduce redundancia deliberada mediante:

- Tablas pre-agregadas y tablas sumario.
- Columnas calculadas y redundancia de claves foráneas.
- Vistas Materializadas (MATERIALIZED VIEW) con refresco periódico.

7.2. Casos de Estudio y Práctica Aplicada

Listing 7.1: Implementación de Vistas Materializadas para desnormalización analítica

```

1  -- Vista materializada que consolida ventas por cliente y reduce
   JOINS recurrentes
2  CREATE MATERIALIZED VIEW mv_customer_sales_summary AS
3  SELECT
4      c.customer_id,
5      c.company_name,
6      c.country,
7      COUNT(DISTINCT o.order_id) AS total_orders,
8      SUM(od.unit_price * od.quantity * (1 - od.discount)) AS
       gross_revenue,
9      MAX(o.order_date) AS last_order_date
10 FROM customers AS c
11 INNER JOIN orders AS o
12     ON c.customer_id = o.customer_id
13 INNER JOIN order_details AS od
14     ON o.order_id = od.order_id
15 GROUP BY
16     c.customer_id,
17     c.company_name,
18     c.country;
19
20 -- Creación de índice único para permitir refrescos
   concurrentes sin bloquear lecturas
21 CREATE UNIQUE INDEX idx_mv_customer_sales_pk ON
   mv_customer_sales_summary (customer_id);
22
23 -- Comando de refresco en segundo plano
24 REFRESH MATERIALIZED VIEW CONCURRENTLY mv_customer_sales_summary;

```

7.3. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Un sistema de e-learning universitario almacena la asignación de profesores, cursos y libros de texto recomendados en una sola tabla, violando la 4FN al asociar profesores con cursos y cursos con libros de forma independiente. Se requiere diseñar el esquema en 4FN y construir una consulta desnormalizada de alta velocidad para el portal estudiantil.

7.3.1. Solución Técnica en PostgreSQL

Listing 7.2: Descomposición en 4FN y consulta consolidada

```

1  -- 1. Descomposición en 4FN (Eliminación de dependencias
    multivaluadas)
2  CREATE TABLE course_professors (
3      course_id VARCHAR(10) NOT NULL,
4      professor_id INTEGER NOT NULL,
5      PRIMARY KEY (course_id, professor_id)
6  );
7
8  CREATE TABLE course_textbooks (
9      course_id VARCHAR(10) NOT NULL,
10     isbn VARCHAR(20) NOT NULL,
11     textbook_title VARCHAR(150) NOT NULL,
12     PRIMARY KEY (course_id, isbn)
13 );
14
15 -- 2. Consulta agregada desnormalizada usando JSONB para el frontend
16 SELECT
17     c.course_id,
18     COALESCE(
19         jsonb_agg(DISTINCT cp.professor_id) FILTER (WHERE
20             cp.professor_id IS NOT NULL),
21         '[]'::jsonb
22     ) AS assigned_professors,
23     COALESCE(
24         jsonb_agg(DISTINCT jsonb_build_object('isbn', ct.isbn,
25             'title', ct.textbook_title))
26         FILTER (WHERE ct.isbn IS NOT NULL),
27         '[]'::jsonb
28     ) AS required_books
29 FROM (SELECT DISTINCT course_id FROM course_professors UNION SELECT
30     DISTINCT course_id FROM course_textbooks) AS c
31 LEFT JOIN course_professors AS cp ON c.course_id = cp.course_id
32 LEFT JOIN course_textbooks AS ct ON c.course_id = ct.course_id
33 GROUP BY
34     c.course_id;

```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Agregación JSONB:** Almacena y proyecta colecciones estructuradas directamente en una sola llamada de base de datos hacia el backend, eliminando problemas de producto cartesiano en clientes HTTP.

Capítulo 8

álgebra Relacional de JOINS, Subconsultas y CTEs

8.1. Marco Conceptual y Teórico (Currículo Universitario)

El álgebra Relacional provee la base matemática de las operaciones de combinación de relaciones.

8.1.1. Familias de Operadores JOIN

- **Inner Join** ($R \bowtie S$): Intersección de tuplas que satisfacen la condición de correspondencia.
- **Left Outer Join** ($R \bowtie_L S$): Preserva todas las tuplas de la relación izquierda R , rellenando con NULL cuando no hay coincidencia en S .
- **Right Outer Join** ($R \bowtie_R S$): Preserva todas las tuplas de la relación derecha S .
- **Full Outer Join** ($R \bowtie_F S$): Preserva todas las tuplas de ambas relaciones.
- **Cross Join** ($R \times S$): Producto cartesiano exhaustivo de todas las tuplas.

8.2. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

El Director de Ventas necesita un reporte que consolide el total facturado por cliente durante el año 1997, incluyendo el nombre de la compañía, país y el número total de órdenes realizadas, ordenado de mayor a menor facturación y asegurando que clientes sin compras aparezcan con monto cero.

8.2.1. Solución Técnica en PostgreSQL con CTEs y LEFT JOIN

Listing 8.1: Consulta modular con CTE y agregación sobre Northwind

```
1 WITH customer_order_totals AS (  
2     SELECT  
3         o.customer_id,  
4         COUNT(DISTINCT o.order_id) AS total_orders,
```

```
5      SUM(od.unit_price * od.quantity * (1 - od.discount)) AS
      total_spent
6  FROM orders AS o
7  INNER JOIN order_details AS od
8      ON o.order_id = od.order_id
9  WHERE EXTRACT(YEAR FROM o.order_date) = 1997
10 GROUP BY
11     o.customer_id
12 )
13 SELECT
14     c.customer_id,
15     c.company_name,
16     c.country,
17     COALESCE(cot.total_orders, 0) AS total_orders_1997,
18     COALESCE(cot.total_spent, 0.00)::NUMERIC(12, 2) AS
        total_revenue_1997
19 FROM customers AS c
20 LEFT JOIN customer_order_totals AS cot
21     ON c.customer_id = cot.customer_id
22 ORDER BY
23     total_revenue_1997 DESC,
24     c.company_name ASC;
```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Estructura con CTE (WITH):** Desacopla la lógica de agregación del detalle de la unión con la tabla de clientes, mejorando la legibilidad y permitiendo al optimizador ejecutar agregaciones tempranas antes del join final.

Capítulo 9

SQL Analítico Avanzado: Funciones de Ventana y Agregación Multidimensional

9.1. Marco Conceptual y Teórico (Currículo Universitario)

Las Funciones de Ventana (*Window Functions*) permiten realizar cálculos sobre un conjunto de filas relacionadas con la fila actual sin colapsar el número de tuplas en el resultado (a diferencia de un `GROUP BY` tradicional).

9.1.1. Sintaxis Formal: Cláusula `OVER()`

```
1 FUNCTION() OVER (  
2     [PARTITION BY col1, col2, ...]  
3     [ORDER BY col_sort ASC|DESC]  
4     [ROWS|RANGE frame_specification]  
5 )
```

9.1.2. Familias de Funciones Analíticas

- **Ranking y Paginación:** `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `NTILE(k)`.
- **Desplazamiento Relativo:** `LAG(col, offset)`, `LEAD(col, offset)`.
- **Agregación Acumulada:** `SUM(col) OVER (...)`, `AVG(col) OVER (...)`.

9.2. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

El área financiera requiere calcular la progresión mensual de ventas acumuladas por categoría de producto durante todo el año histórico y el ranking de ventas de cada producto respecto a su categoría en el mismo mes.

9.2.1. Solución Técnica en PostgreSQL (Formato O'Reilly)

Listing 9.1: Consulta analítica con Window Functions y acumulados

```

1 WITH monthly_product_revenue AS (
2     SELECT
3         p.category_id,
4         c.category_name,
5         p.product_name,
6         DATE_TRUNC('month', o.order_date)::DATE AS sales_month,
7         SUM(od.unit_price * od.quantity * (1 - od.discount)) AS
            total_revenue
8     FROM orders AS o
9     INNER JOIN order_details AS od
10        ON o.order_id = od.order_id
11     INNER JOIN products AS p
12        ON od.product_id = p.product_id
13     INNER JOIN categories AS c
14        ON p.category_id = c.category_id
15     GROUP BY
16         p.category_id,
17         c.category_name,
18         p.product_name,
19         DATE_TRUNC('month', o.order_date)::DATE
20 )
21 SELECT
22     category_name,
23     product_name,
24     sales_month,
25     total_revenue::NUMERIC(10, 2) AS monthly_sales,
26     -- Acumulado m'ovil anual por producto
27     SUM(total_revenue) OVER (
28         PARTITION BY category_name, product_name
29         ORDER BY sales_month
30     )::NUMERIC(12, 2) AS running_total_ytd,
31     -- Ranking mensual dentro de la categor'ia
32     DENSE_RANK() OVER (
33         PARTITION BY category_name, sales_month
34         ORDER BY total_revenue DESC
35     ) AS category_monthly_rank
36 FROM monthly_product_revenue
37 ORDER BY
38     category_name ASC,
39     sales_month ASC,
40     category_monthly_rank ASC;

```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **DENSE_RANK() vs RANK():** **DENSE_RANK()** asigna rangos consecutivos sin saltos numéricos en caso de empates de montos (1,2,2,3...), ideal para tablas de clasificación.

Capítulo 10

Programación en Base de Datos: PL/pgSQL, Procedimientos y Triggers

10.1. Marco Conceptual y Teórico (Currículo Universitario)

La arquitectura procedimental en servidor permite ejecutar bloques de código complejos con control transaccional, reduciendo los saltos de red (*network roundtrips*) entre la aplicación y el motor de base de datos.

10.1.1. Estructura de Bloques en PL/pgSQL

Todo bloque procedimental se define mediante la sintaxis delimitada:

```
1 [ <<label>> ]
2 [ DECLARE
3     declarations ]
4 BEGIN
5     statements
6 [ EXCEPTION
7     WHEN condition THEN
8         handler_statements ]
9 END [ label ];
```

10.1.2. Disparadores (Triggers) y Eventos

Un trigger es una función que se ejecuta automáticamente cuando ocurre un evento de modificación de datos (INSERT, UPDATE, DELETE o TRUNCATE):

- **Nivel de Fila (FOR EACH ROW):** Se ejecuta por cada registro afectado. Tiene acceso a las variables de contexto pseudo-registro NEW y OLD.
- **Nivel de Sentencia (FOR EACH STATEMENT):** Se ejecuta una sola vez por cada instrucción SQL, independientemente del número de filas modificadas.
- **Tiempos de Disparo:** BEFORE (para validaciones o transformaciones previas), AFTER (para registros de auditoría y cascadas) o INSTEAD OF (sobre vistas).

10.2. Casos de Estudio y Práctica Aplicada

Listing 10.1: Procedimiento almacenado con transacción y validación de saldo

```
1 CREATE OR REPLACE PROCEDURE transfer_balance(  
2     p_sender_id BIGINT,  
3     p_receiver_id BIGINT,  
4     p_amount NUMERIC(12, 2)  
5 )  
6 LANGUAGE plpgsql  
7 AS $$  
8 DECLARE  
9     v_sender_balance NUMERIC(12, 2);  
10 BEGIN  
11     -- Validar monto  
12     IF p_amount <= 0 THEN  
13         RAISE EXCEPTION 'El monto a transferir debe ser mayor a cero.  
14             Monto ingresado: %', p_amount;  
15     END IF;  
16  
17     -- Bloqueo pesimista de cuenta emisora  
18     SELECT balance INTO v_sender_balance  
19     FROM bank_accounts  
20     WHERE account_id = p_sender_id  
21     FOR UPDATE;  
22  
23     IF NOT FOUND THEN  
24         RAISE EXCEPTION 'Cuenta emisora ID % no encontrada',  
25             p_sender_id;  
26     END IF;  
27  
28     IF v_sender_balance < p_amount THEN  
29         RAISE EXCEPTION 'Saldo insuficiente. Saldo actual: %,  
30             Requerido: %', v_sender_balance, p_amount;  
31     END IF;  
32  
33     -- D\debito y Cr\credito  
34     UPDATE bank_accounts SET balance = balance - p_amount WHERE  
35         account_id = p_sender_id;  
36     UPDATE bank_accounts SET balance = balance + p_amount WHERE  
37         account_id = p_receiver_id;  
38  
39     COMMIT;  
40 END;  
41 $$;
```

10.3. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Implementar un sistema de auditoría estricto en PostgreSQL para la tabla `products` de Northwind. Cada vez que el precio unitario de un producto sea modificado, el sistema debe registrar en una tabla de auditoría el ID del producto, el precio anterior, el nuevo precio, la diferencia porcentual, el usuario que ejecutó la acción y la marca de tiempo exacta, impidiendo incrementos superiores al 50 % de un solo cambio.

10.3.1. Solución Técnica con Trigger en PostgreSQL

Listing 10.2: Trigger con validación de umbral y auditoría

```

1  -- 1. Tabla de Registro de Auditoría de Precios
2  CREATE TABLE product_price_audit (
3      audit_id BIGSERIAL PRIMARY KEY,
4      product_id SMALLINT NOT NULL,
5      old_unit_price REAL NOT NULL,
6      new_unit_price REAL NOT NULL,
7      percentage_change NUMERIC(6, 2) NOT NULL,
8      changed_by VARCHAR(80) DEFAULT CURRENT_USER,
9      changed_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
10 );
11
12 -- 2. Función de Trigger con Regla de Negocio
13 CREATE OR REPLACE FUNCTION trg_validate_and_audit_price()
14 RETURNS TRIGGER
15 LANGUAGE plpgsql
16 AS $$
17 DECLARE
18     v_pct_change NUMERIC(6, 2);
19 BEGIN
20     -- Solo actuar si el precio efectivamente cambió
21     IF NEW.unit_price <> OLD.unit_price THEN
22         -- Calcular porcentaje de cambio
23         v_pct_change := ((NEW.unit_price - OLD.unit_price) /
24             OLD.unit_price) * 100.0;
25
26         -- Regla de Negocio: Bloquear incrementos mayores al 50%
27         IF v_pct_change > 50.0 THEN
28             RAISE EXCEPTION 'Operación rechazada: Incremento de
29                 precio del % supera el umbral permitido del 50%% para
30                 el producto ID %',
31                 ROUND(v_pct_change, 2), OLD.product_id;
32         END IF;
33
34         -- Insertar en auditoría
35         INSERT INTO product_price_audit (
36             product_id,
37             old_unit_price,
38             new_unit_price,
39             percentage_change
40         ) VALUES (
41             OLD.product_id,
42             OLD.unit_price,
43             NEW.unit_price,

```

```
41         v_pct_change
42     );
43     END IF;
44
45     RETURN NEW;
46 END;
47 $$;
48
49 -- 3. Asignación del Trigger a la Tabla
50 CREATE TRIGGER trigger_audit_product_price
51 BEFORE UPDATE OF unit_price ON products
52 FOR EACH ROW
53 EXECUTE FUNCTION trg_validate_and_audit_price();
```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **BEFORE UPDATE OF column:** Al especificar la columna objetivo en el evento (OF unit_price), PostgreSQL optimiza el rendimiento evitando invocar la función del trigger cuando se actualizan otras columnas como el stock o la descripción.

Parte III

Data Warehousing, Rendimiento y Alta Disponibilidad

Capítulo 11

Data Warehousing, Modelado Dimensional y Arquitectura OLAP

11.1. Marco Conceptual y Teórico (Currículo Universitario)

El Data Warehouse corporativo almacena datos históricos integrados desde múltiples fuentes operacionales heterogéneas para alimentar herramientas analíticas de Business Intelligence (BI).

11.1.1. Comparativa OLTP vs OLAP

Dimensión	Sistemas OLTP (Operacionales)	Sistemas OLAP (Analíticos)
Propósito	Procesamiento de transacciones en tiempo real	Soporte a decisiones y análisis histórico
Diseño	Altamente normalizado (3FN / BCNF)	Desnormalizado (Estrella / Copo de Nieve)
Patrón I/O	Millones de escrituras cortas (INSERT/UPDATE)	Consultas complejas de lectura masiva
Métrica	Rendimiento de transacciones por segundo (TPS)	Tiempo de respuesta de consultas analíticas

Tabla 11.1: Diferencias estructurales entre sistemas OLTP y OLAP

11.1.2. Metodología Kimball: Hechos y Dimensiones

- **Tablas de Hechos (*Fact Tables*):** Centralizan los eventos cuantitativos del negocio (montos, cantidades, duraciones). Contienen claves foráneas hacia las dimensiones y medidas aditivas, semi-aditivas o no aditivas.
- **Tablas de Dimensiones (*Dimension Tables*):** Proporcionan el contexto descriptivo (*quién, qué, dónde, cuándo*).
- **Dimensiones de Cambio Lento (SCD):**
 - *SCD Tipo 1:* Sobrescribe el valor anterior (sin historial).
 - *SCD Tipo 2:* Crea un nuevo registro con fechas de vigencia (`start_date`, `end_date`, `is_current`).
 - *SCD Tipo 3:* Almacena el valor anterior en una columna separada.

11.2. Casos de Estudio: Operaciones OLAP (CUBE y ROLLUP)

Listing 11.1: Agregación multidimensional jerárquica con ROLLUP

```

1  -- Consulta analítica con totales y subtotales automáticos por
    jerarquía temporal
2  SELECT
3      EXTRACT(YEAR FROM o.order_date)::INTEGER AS sales_year,
4      EXTRACT(QUARTER FROM o.order_date)::INTEGER AS sales_quarter,
5      p.category_id,
6      SUM(od.unit_price * od.quantity * (1 - od.discount))::NUMERIC(14,
7          2) AS total_revenue
8  FROM orders AS o
9  INNER JOIN order_details AS od ON o.order_id = od.order_id
10 INNER JOIN products AS p ON od.product_id = p.product_id
11 GROUP BY
12     ROLLUP (
13         EXTRACT(YEAR FROM o.order_date),
14         EXTRACT(QUARTER FROM o.order_date),
15         p.category_id
16     )
17 ORDER BY
18     sales_year NULLS FIRST,
19     sales_quarter NULLS FIRST,
20     p.category_id NULLS FIRST;

```

11.3. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Diseñar e implementar el esquema físico de Data Warehouse para el área de Recursos Humanos de una multinacional, modelando la dimensión de tiempo, departamento y empleados con claves subrogadas, y la tabla de hechos con costos totales y conteo de personal.

11.3.1. Solución Técnica DDL Dimensional

Listing 11.2: Esquema estrella DDL para Data Warehouse de RRHH

```

1  -- 1. Dimensión Empleado (Con Clave Subrogada)
2  CREATE TABLE dim_employee (
3      employee_key SERIAL PRIMARY KEY,
4      employee_id INTEGER NOT NULL,
5      full_name VARCHAR(100) NOT NULL,
6      job_title VARCHAR(50) NOT NULL,
7      salary NUMERIC(10, 2) NOT NULL,
8      valid_from DATE NOT NULL DEFAULT CURRENT_DATE,
9      valid_to DATE,
10     is_current BOOLEAN DEFAULT TRUE
11 );
12
13 -- 2. Dimensión Departamento
14 CREATE TABLE dim_department (

```

```

15     department_key SERIAL PRIMARY KEY,
16     department_id INTEGER NOT NULL,
17     department_name VARCHAR(60) NOT NULL,
18     city VARCHAR(50) NOT NULL,
19     country VARCHAR(50) NOT NULL
20 );
21
22 -- 3. Dimensi\on Tiempo
23 CREATE TABLE dim_date (
24     date_key INTEGER PRIMARY KEY, -- Formato entero YYYYMMDD
25     full_date DATE NOT NULL UNIQUE,
26     month_number SMALLINT NOT NULL,
27     month_name VARCHAR(15) NOT NULL,
28     quarter_number SMALLINT NOT NULL,
29     year_number SMALLINT NOT NULL
30 );
31
32 -- 4. Tabla de Hechos: N\omina y Planilla
33 CREATE TABLE fact_payroll (
34     payroll_fact_id BIGSERIAL PRIMARY KEY,
35     employee_key INTEGER NOT NULL REFERENCES
36         dim_employee(employee_key),
37     department_key INTEGER NOT NULL REFERENCES
38         dim_department(department_key),
39     date_key INTEGER NOT NULL REFERENCES dim_date(date_key),
40     total_cost NUMERIC(12, 2) NOT NULL,
41     headcount_contribution SMALLINT DEFAULT 1
42 );
43
44 -- \indices B-Tree en claves for\aneas para acelerar JOINS
45     dimensionales
46 CREATE INDEX idx_fact_payroll_emp ON fact_payroll (employee_key);
47 CREATE INDEX idx_fact_payroll_dept ON fact_payroll (department_key);
48 CREATE INDEX idx_fact_payroll_date ON fact_payroll (date_key);

```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Claves Subrogadas (*Surrogate Keys*):** Se emplean enteros autoincrementales (`employee_key`) en lugar de claves de negocio operacionales (`employee_id`) para permitir el seguimiento histórico de cambios mediante SCD Tipo 2 e independizar el almacén analítico de los sistemas transaccionales fuente.

Capítulo 12

Optimización de Consultas, Planes de Ejecución e índices

12.1. Marco Conceptual y Teórico (Currículo Universitario)

El optimizador de consultas basado en costos (*Cost-Based Optimizer / CBO*) de PostgreSQL genera múltiples planes de ejecución y selecciona aquel con el menor costo estimado en unidades arbitrarias de acceso a disco y CPU.

12.1.1. Componentes de un Plan de Ejecución

- **Sequential Scan (Seq Scan):** Lectura completa de todas las páginas de la tabla. Eficiente para tablas pequeñas o cuando la consulta recupera un alto porcentaje del total de filas (> 20 %).
- **Index Scan:** Búsqueda en la estructura B-Tree del índice seguida de la lectura de las páginas de datos en el Heap.
- **Index Only Scan:** Recuperación de datos directamente desde las hojas del índice sin acceder al Heap (requiere que el Visibility Map confirme que la página no tiene tuplas modificadas pendientes).
- **Bitmap Index Scan + Bitmap Heap Scan:** Combina múltiples índices generando un mapa de bits en memoria antes de leer el Heap secuencialmente por bloques.

12.1.2. Estrategias de Indexación Avanzada

1. **índices Compuestos:** Creados sobre múltiples columnas (`col1`, `col2`). Siguen la regla del prefijo izquierdo.
2. **índices Parciales:** `CREATE INDEX ... WHERE condicion`; optimizan espacio e indexan únicamente filas relevantes (ej. `WHERE status = 'PENDIENTE'`).
3. **índices Expresionales:** `CREATE INDEX ... ON users(LOWER(email));`.
4. **índices Cubrientes (INCLUDE):** Añaden columnas al nivel de hoja sin incluirlas en la clave de ordenamiento del B-Tree.

—

12.2. Casos de Estudio y Práctica Aplicada

Listing 12.1: Análisis comparativo de costos con EXPLAIN ANALYZE

```

1  -- 1. Inspecci\on de consulta sin \indice adecuado
2  EXPLAIN (ANALYZE, BUFFERS, TIMING, FORMAT TEXT)
3  SELECT order_id, customer_id, order_date
4  FROM orders
5  WHERE customer_id = 'ALFKI'
6         AND order_date >= '1997-01-01';
7
8  -- 2. Creaci\on de \indice compuesto cubriente
9  CREATE INDEX idx_orders_customer_date_inc
10 ON orders (customer_id, order_date)
11 INCLUDE (order_id);
12
13 -- 3. Re-evaluaci\on del plan de ejecuci\on (Transformaci\on a
    Index Only Scan)
14 EXPLAIN (ANALYZE, BUFFERS, TIMING, FORMAT TEXT)
15 SELECT order_id, customer_id, order_date
16 FROM orders
17 WHERE customer_id = 'ALFKI'
18        AND order_date >= '1997-01-01';

```

12.3. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Una consulta crítica del panel administrativo de ventas experimenta degradación de rendimiento severa al escanear secuencialmente millones de registros en la tabla de detalle de órdenes. Se requiere identificar cuellos de botella mediante EXPLAIN, proponer la estrategia de indexación y verificar la reducción del costo computacional.

12.3.1. Solución Técnica y Estrategia de Tuning

Listing 12.2: Optimización de consulta analítica y creación de índice GIN / Parcial

```

1  -- Consulta objetivo: B\usqueda de \ordenes de alto volumen con
    descuento mayor al 15%
2  EXPLAIN (ANALYZE, BUFFERS)
3  SELECT
4      order_id,
5      product_id,
6      unit_price,
7      quantity,
8      discount
9  FROM order_details
10 WHERE discount > 0.15
11 ORDER BY quantity DESC;
12
13 -- Estrategia: \indice parcial con ordenamiento invertido en cantidad
14 CREATE INDEX idx_order_details_discount_qty
15 ON order_details (quantity DESC)
16 WHERE discount > 0.15;

```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Impacto de Buffers (shared hit):** La métrica `Buffers: shared hit=N read=M` indica cuántos bloques de 8KB fueron leídos directamente desde la memoria RAM (*Shared Buffers*) versus disco físico. Una optimización exitosa maximiza el ratio de aciertos (`hit percent > 99%`).

Capítulo 13

Alta Disponibilidad (HA), NoSQL y Arquitecturas Cloud Modernas

13.1. Marco Conceptual y Teórico (Currículo Universitario)

La arquitectura de datos contemporánea exige tolerancia a fallos, recuperación ante desastres y escalabilidad horizontal tanto para cargas estructuradas como semiestructuradas.

13.1.1. El Teorema CAP y Bases NoSQL

Formulado por Eric Brewer, establece que en un sistema distribuido con particionamiento de red (**Partition Tolerance - P**), es imposible garantizar simultáneamente:

- **Consistencia Fuerte (C):** Todos los nodos devuelven la misma versión más reciente del dato en todo momento.
- **Disponibilidad (A):** Toda petición no fallida recibe una respuesta garantizada.

13.1.2. Arquitecturas de Alta Disponibilidad en RDBMS

- **Streaming Replication (PostgreSQL):** Transmisión continua de registros WAL desde un nodo primario (*Read/Write*) hacia réplicas en espera (*Standby / Read Only*) de forma síncrona o asíncrona.
- **Grupos de Disponibilidad AlwaysOn (SQL Server):** Replicación a nivel de base de datos con conmutación automática por error (*Automatic Failover*).
- **Connection Pooling (PgBouncer):** Gestión y reutilización de conexiones para evitar la saturación de procesos en el servidor.

13.2. Casos de Estudio: Almacenamiento Semiestructurado con JSONB y GIN

Listing 13.1: Modelado híbrido relacional/documental en PostgreSQL

```
1 -- Tabla h\`ibrida con metadatos no estructurados en formato binario
   JSONB
2 CREATE TABLE device_telemetry (
3     telemetry_id BIGSERIAL PRIMARY KEY,
```

```

4     device_id VARCHAR(50) NOT NULL,
5     recorded_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
6     metrics JSONB NOT NULL
7 );
8
9 -- \ 'índice GIN para b\ 'usquedas ultra-r\ 'apidas dentro del documento
   JSONB
10 CREATE INDEX idx_telemetry_metrics_gin ON device_telemetry USING GIN
   (metrics);
11
12 -- Consulta con operador de contenci\ 'on (@>)
13 SELECT
14     device_id,
15     recorded_at,
16     metrics->>'temperature' AS temp_celsius,
17     metrics->>'network'->>'ip_address' AS client_ip
18 FROM device_telemetry
19 WHERE metrics @> '{"status": "CRITICAL", "sensor_type": "thermal"}';

```

13.3. Taller de Consolidación del Capítulo

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Una plataforma de telemetría IoT vehicular requiere almacenar registros de diagnósticos con estructuras de datos variables según el fabricante, permitiendo indexar propiedades internas del JSONB y consultar métricas de fallo en milisegundos.

13.3.1. Solución Técnica con índices de Expresión y GIN

Listing 13.2: Indexación especializada sobre campos JSONB anidados

```

1 -- 1. Inserci\ 'on de telemetr\ 'ia vehicular
2 INSERT INTO device_telemetry (device_id, metrics)
3 VALUES (
4     'VIN-98234-A',
5     '{
6         "status": "CRITICAL",
7         "sensor_type": "thermal",
8         "temperature": 105.4,
9         "engine": {"rpm": 4200, "oil_pressure_psi": 18.2},
10        "error_codes": ["P0300", "P0117"]
11    }'::jsonb
12 );
13
14 -- 2. \ 'índice de expresi\ 'on B-Tree para valores num\ 'ericos
   extra\ 'idos
15 CREATE INDEX idx_telemetry_temp_num ON device_telemetry (
16     ((metrics->>'temperature')::NUMERIC)
17 ) WHERE metrics ? 'temperature';
18
19 -- 3. Consulta optimizada de alerta
20 SELECT
21     device_id,
22     recorded_at,

```

```
23 (metrics->>'temperature')::NUMERIC AS temp
24 FROM device_telemetry
25 WHERE metrics ? 'temperature'
26 AND (metrics->>'temperature')::NUMERIC > 100.0;
```

[BUENAS PRACTICAS] Desglose Técnico y Buenas Prácticas

- **Ventaja de JSONB frente a JSON:** JSONB descompone y almacena el documento en formato binario pre-parseado, eliminando espacios redundantes y soportando indexación GIN que acelera consultas de contención en órdenes de magnitud comparado con búsquedas de texto.

Parte IV

Laboratorios de Consolidación e Infraestructura

Capítulo 14

Infraestructura Cloud, Despliegue en AWS EC2 y Benchmark Northwind

14.1. Aprovisionamiento de Infraestructura en AWS EC2

Para crear un entorno reproducible de grado empresarial, se orquesta un contenedor con PostgreSQL 16 sobre una instancia EC2 (Amazon Linux 2023 / Ubuntu).

14.1.1. Manifiesto de Orquestación Docker Compose

Listing 14.1: Manifiesto docker-compose.yml de producción

```
version: '3.8'

services:
  postgres-db:
    image: postgres:16-alpine
    container_name: postgres-production
    restart: unless-stopped
    environment:
      POSTGRES_DB: northwind
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${DB_PASSWORD:-postgres_secure_2026}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db_northwind.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d northwind"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  pgdata:
    driver: local
```

14.2. Batería de Ejercicios Graduales y Solucionario Técnico

[CASO EMPRESARIAL] Enunciado de Problema de Negocio

Reto de Entrevista Senior Data Engineer: Obtener los 3 mejores clientes de cada país según el total gastado históricamente, considerando únicamente pedidos entregados (`shipped_date IS NOT NULL`).

14.2.1. Solución Técnica en PostgreSQL

Listing 14.2: Solución con CTE y partición analítica por país

```
1 WITH customer_country_spending AS (  
2     SELECT  
3         c.country,  
4         c.customer_id,  
5         c.company_name,  
6         SUM(od.unit_price * od.quantity * (1 - od.discount)) AS  
7             total_spent,  
8         DENSE_RANK() OVER (  
9             PARTITION BY c.country  
10            ORDER BY SUM(od.unit_price * od.quantity * (1 -  
11                od.discount)) DESC  
12        ) AS country_rank  
13 FROM customers AS c  
14 INNER JOIN orders AS o  
15     ON c.customer_id = o.customer_id  
16 INNER JOIN order_details AS od  
17     ON o.order_id = od.order_id  
18 WHERE o.shipped_date IS NOT NULL  
19 GROUP BY  
20     c.country,  
21     c.customer_id,  
22     c.company_name  
23 )  
24 SELECT  
25     country,  
26     country_rank,  
27     customer_id,  
28     company_name,  
29     total_spent::NUMERIC(12, 2) AS total_spent  
30 FROM customer_country_spending  
31 WHERE country_rank <= 3  
32 ORDER BY  
33     country ASC,  
34     country_rank ASC;
```

Capítulo 15

Taller Multi-Motor y Casos de Negocio (PostgreSQL / MySQL)

15.1. Desarrollo Práctico Multi-Motor y Compatibilidad

A lo largo del diseño de bases de datos, los ingenieros deben dominar las particularidades sintácticas entre los dos motores de código abierto más utilizados del mundo: **PostgreSQL** y **MySQL**.

15.1.1. Comparativa de Sintaxis y Dialectos

Característica	PostgreSQL 16+	MySQL 8.0+
Autoincrementables	SERIAL / IDENTITY	AUTO_INCREMENT
Manejo de Conflictos	ON CONFLICT ... DO UPDATE	ON DUPLICATE KEY UPDATE
Retorno de inserciones	Cláusula RETURNING	No soportado nativamente
Tipado Booleano	Nativo (BOOLEAN)	Alias de TINYINT(1)
JSON Avanzado	JSONB con índices GIN	JSON binario

Tabla 15.1: Diferencias clave de sintaxis entre PostgreSQL y MySQL

15.2. Casos de Negocio Reales

15.2.1. Caso 1: Cineplex (Gestión de Salas y Boletos)

Listing 15.1: Consulta de recaudación por sala de cine

```
1 SELECT
2     c.cinema_name ,
3     ms.movie_title ,
4     COUNT(b.booking_id) AS tickets_sold ,
5     SUM(ms.ticket_price) AS gross_revenue
6 FROM cinemas AS c
7 INNER JOIN movie_showtimes AS ms
8     ON c.cinema_id = ms.cinema_id
9 LEFT JOIN bookings AS b
10     ON ms.showtime_id = b.showtime_id
11 GROUP BY
12     c.cinema_name ,
13     ms.movie_title
```

```
14 ORDER BY
15     gross_revenue DESC NULLS LAST;
```